

Слайд 1.

Здравствуйте, меня зовут Номеровский Максим Сергеевич и я хочу представить вам свою курсовую работу на тему РАЗРАБОТКА МЕТОДИКИ ИССЛЕДОВАНИЯ СООТВЕТСТВИЯ ОС LINUX ЗАДАЧАМ РЕАЛЬНОГО ВРЕМЕНИ. Актуальность моей работы заключается в том, что в современное время появляется всё больше устройств, которым необходимо выполнять какие-либо задачи в режиме данного времени.

Слайд 2.

Целью работы является разработка методики исследования соответствия операционной системы Linux задачам реального времени на конкретной аппаратной платформе и проведения сравнительного анализа при использовании ядра с активированным параметром PREEMPT_RT и без него для решения простейших задач реального времени. Для достижения поставленной цели я решил следующие задачи:

1. Выбрать и изучить аппаратную платформу, на которой будет проверяться и тестироваться методика.
2. Выполнить кросс-компиляцию ядра Linux с параметрами, удовлетворяющими техническим требованиям выбранной аппаратной платформы, в двух конфигурациях (с поддержкой реального времени и без нее).
3. Подготовить загрузочный образ системы, включающий загрузчик U-Boot, скомпилированное ядро, дерево устройств и корневую файловую систему.
4. Разработать программно-аппаратный комплекс для генерации тестовых импульсов, фиксации задержек отклика и автоматизированного сбора статистики.
5. Собрать тестовый стенд и провести серию аппаратных и программных измерений метрик реального времени в различных конфигурациях, в том числе с имитацией сторонней нагрузки на процессор.

Слайд 3.

Для выполнения данной работы мной было использовано такое программное обеспечение как Arduino, ядро Linux и язык программирования Python.

Слайд 4.

В ходе работы для исследования были разработаны программное обеспечение, которое делится на 3 компонента:

1. Генератор меандра для платформы Arduino. Данное программное обеспечение генерирует прямоугольные импульсы длительностью 250 мс и подает их на входной порт MangoPi MQ Pro, а после получения ответа, отправляет данные о задержке на ПК.
2. Программа-обработчик входящих импульсов. Программа функционирует на целевой плате MangoPi MQ Pro и осуществляет опрос входного порта (GPIO). При регистрации изменений на нём, плата начинает выдавать на выход сигнал с тем же значением, что и пришёл на неё.
3. Программа сбора и анализа данных на ПК. Приложение выполняется на ПК, принимает входящий поток данных от контроллера Arduino, производит вычисление средних значений задержек и показателей джиттера и после этого сохраняет собранные значения в csv файл.

Слайд 5.

На данном слайде представлена блок-схема работы разработанного программного обеспечения. Сначала инициализируется программа на ПК и готовится io поток для сохранения данных в csv файл, затем запускается цикл измерений на Arduino. При каждой итерации цикла программа на ПК проверяет собрали ли мы нужное количество данных для анализа. Если да, то программа завершается и файл csv сохраняется, если нет, запускается отправка сигнала на плату Arduino. После отправки программа на Mango ловит сигнал и начинает отправлять его в ответ. Arduino принимает его, фиксирует время ответа и ждёт 250мкс. Затем Arduino перестаёт передавать сигнал и ожидает ответной реакции от Mango. В

конце итерации Arduino также фиксирует время ответа и отправляет данные на ПК. ПК же в свою очередь принимает данные и считает среднюю задержку за все собранные эксперименты, а затем записывает результат в файл.

Слайд 6.

Сначала для тестирования разработанной методики я использовал аппаратный метод измерений, т. е. через осциллограф. Мной были использованы данные конфигурации системы при тестировании.

1. С использованием ядра Linux без параметра PREEMPT_RT.
2. С использованием ядра Linux без параметра PREEMPT_RT с наивысшим приоритетом у программы-ответчика.
3. С использованием ядра Linux с параметром PREEMPT_RT.
4. С использованием ядра Linux с параметром PREEMPT_RT с наивысшим приоритетом у программы-ответчика.

Слайд 7.

Результаты аппаратных измерений представлены в данной таблице. Анализ полученных данных показывает, что минимальная задержка реакции на получение входного сигнала наблюдается в конфигурациях 2 и 3. Однако наименьшая задержка реакции на прекращение сигнала зафиксирована в конфигурации 4. Таким образом, оптимальной конфигурацией с точки зрения совокупной скорости обработки является конфигурация 4.

Слайд 8.

Далее мной были проведены тесты методики с использованием разработанной ранее программы. В данных тестах мной были добавлены конфигурации с использованием утилиты stress, которая будет давать дополнительную нагрузку на плату Mango.

1. С использованием ядра Linux без параметра PREEMPT_RT.
2. С использованием ядра Linux без параметра PREEMPT_RT с утилитой stress.
3. С использованием ядра Linux без параметра PREEMPT_RT с наивысшим приоритетом у программы-ответчика.
4. С использованием ядра Linux без параметра PREEMPT_RT с наивысшим приоритетом у программы-ответчика с утилитой stress.
5. С использованием ядра Linux с параметром PREEMPT_RT.
6. С использованием ядра Linux с параметром PREEMPT_RT с утилитой stress.
7. С использованием ядра Linux с параметром PREEMPT_RT с наивысшим приоритетом у программы-ответчика.
8. С использованием ядра Linux с параметром PREEMPT_RT с наивысшим приоритетом у программы-ответчика с утилитой stress.

Слайд 9.

Результаты программных измерений представлены на слайде. Анализ тестирования конфигураций ОС показал, что **назначение высшего приоритета** критическим задачам является главным фактором снижения задержек и защиты от фоновой нагрузки. Из-за архитектурных накладных расходов ядро **PREEMPT_RT** демонстрирует более высокие *средние* задержки по сравнению со стандартным. Однако при правильной настройке именно RT-ядро эффективнее ограничивает **пиковые задержки** (наихудшее время отклика), что делает его критически важным для систем жесткого реального времени.

Слайд 10.

Таким образом в ходе исследований мной были получены следующие результаты:

1. Осуществлена кросс-компиляция двух версий ядра Linux (стандартной и с интегрированным параметром PREEMPT_RT).

2. Подготовлен загрузочный образ операционной системы, включающий системный загрузчик U-Boot и корневую файловую систему.
3. Спроектирован и собран специализированный аппаратно-программный стенд с использованием микроконтроллера Arduino для высокоточной генерации тестовых импульсов и фиксации задержек отклика.
4. Было установлено, что наличие патча PREEMPT_RT не увеличивает среднюю пропускную способность системы; напротив, из-за дополнительных накладных расходов на переключение контекста средняя длительность задержки может возрастать.
5. Экспериментально доказано, что при жестком распределении приоритетов в режиме SCHED_FIFO ядро реального времени эффективно изолирует процессы от фоновой нагрузки. В условиях стресс-тестирования конфигурация с PREEMPT_RT показала значительное преимущество над стандартным ядром Linux.